Secured Doors

Design 1: Abstract class SecuredDoor with three subclasses

Each door type (AluminiumDoor, IronDoor, TitaniumDoor) extends an abstract base class SecuredDoor. The common logic (clearance checking, unlock action) is in SecuredDoor, and each subclass only overrides requiredClearance() and unlockEffect().

Pros	Cons
Following the DRY principle, the common door logic is centralized in `SecuredDoor` and is not repeated across the three classes.	It requires additional abstract classes, which increases the complexity of the class structure.
Following the OCP principle, adding a new door type requires only creating a new subclass; the existing code does not need to be changed.	
Following the SRP principle, each class defines only its own unique unlocking effects.	
Following the DIP principle, `UnlockDoorAction` depends on the abstract class `SecuredDoor` rather than a specific door class.	
Has Low connascence of algorithm because subclasses don't need to know how clearance checking works internally only how to define their own effect resulting in loose coupling.	

Design 2: Three independent door types

Each door type directly inherits from `Ground` and implements its own access card verification and unlocking logic.

Pros	Cons
The structure is simpler and does not require abstract classes.	This violates the DRY principle, as the access card verification logic is repeated across three classes.
Each class is completely independent, making it easier to understand when read on its own.	This violates the OCP principle,adding new door types requires copying and changing existing door code.
	This violates the DIP principle; `UnlockDoorAction` needs to depend on each specific door type individually.

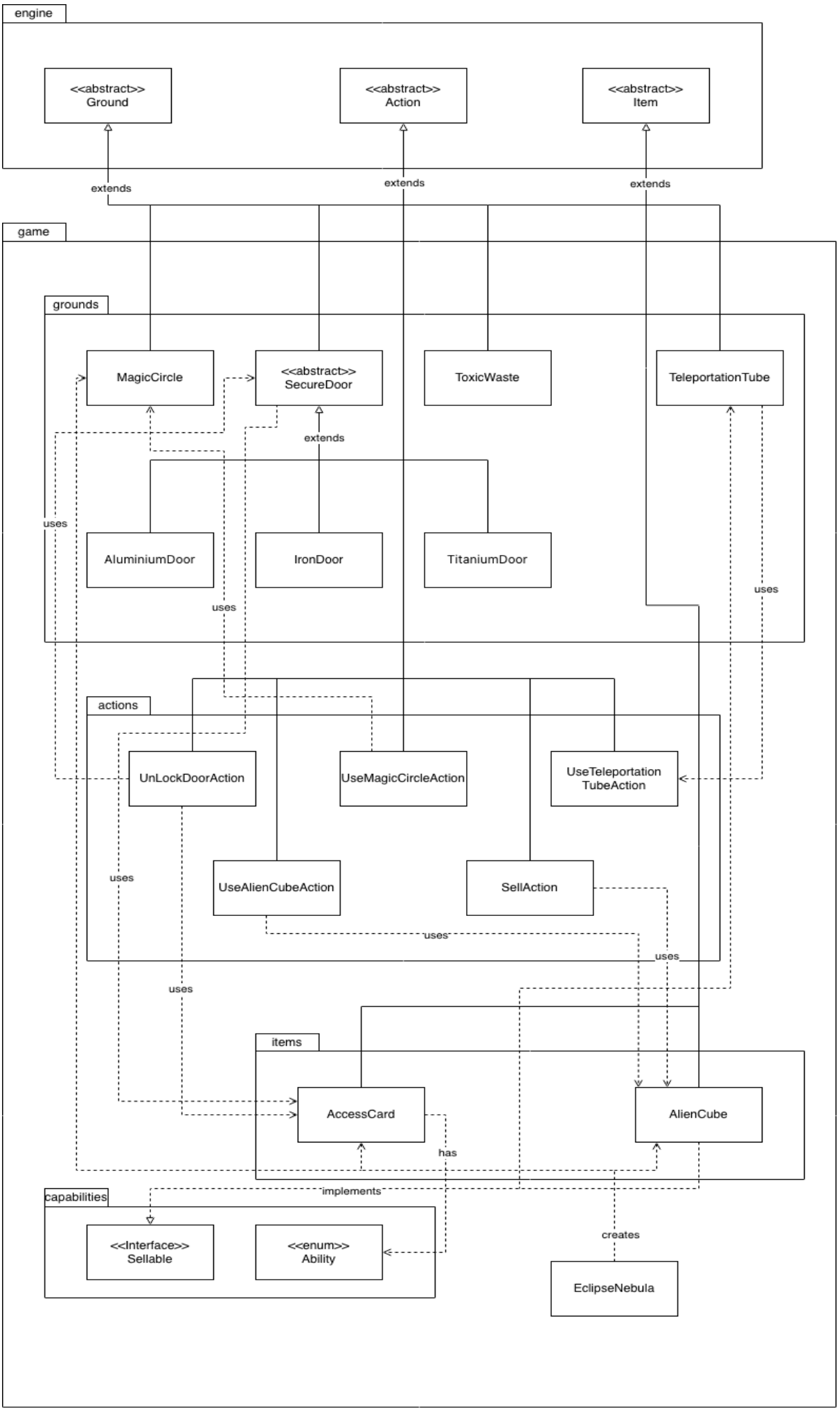
Teleportation Design

Design 1: Separate the teleportation per device.

Pros	Cons
Follows SRP because each action class is solely responsible for the teleportation logic of its own device.	There are more classes to navigate.
Follows OCP because adding a new teleportation device on requires creating a new action class and no existing teleportation logic needs to be altered.	
Follows KISS because each action class is small and focused.	
Low connascence of algorithm because each device's teleportation logic is independent with no action class needing to know how another device teleports, therefore loose coupling.	

Design 2: Shared TeleportAction base class for all devices.

Pros	Cons
Follows DRY which is where common teleportation logic such as moving the actor and finding a destination is centralised.	Introduces an additional abstract class adding a layer to the class hierarchy.
	Does not follow kiss because forcing three different teleportation mechanisms into a shared hierarchy adds unnecessary complexity.
	High connascence of algorithms because all teleportation actions become coupled to the shared base class meaning that changes to the base class affect all devices.


engine
<<abstract>>
Ground
<<abstract>>
Action
<<abstract>>
Item
extends
extends
extends
game
grounds
MagicCircle
<<abstract>>
SecureDoor
ToxicWaste
TeleportationTube
extends
uses
AluminiumDoor
IronDoor
TitaniumDoor
uses
uses
actions
UnLockDoorAction
UseMagicCircleAction
UseTeleportation
TubeAction
uses
UseAlienCubeAction
SellAction
uses
uses
uses
items
AccessCard
AlienCube
has
implements
capabilities
<<Interface>>
Sellable
<<enum>>
Ability
creates
EclipseNebula

We choose design 1 for both secured doors and teleportation design. REQ2 shown in the UML implemented tiered security doors requiring access cards and three distinct teleportation devices, each with unique side effects. For Secured Doors we choose design 1 because adding in the abstract class ensures that each subclass only overrides unlockEffect(), keeping individual classes small and focused (KISS). UnlockDoorAction depends solely on the SecuredDoor abstraction meaning new door types can be introduced by creating a new subclass meeting OCP and DIP. Separating doors by each unlock effect in subclasses ensures connascence of algorithm is kept low as no door class needs to know how another door's clearance checking works internally.

For the teleportation design, although design 2 would centralise common teleportation logic following DRY, the three devices are very different in how they select destinations and apply side effects. Therefore we choose design 1 to ensure that KISS is not violated through the shared hierarchy which adds complexity. By keeping each class independent, connascence of algorithm is kept low as no teleportation action needs to know how other devices work internally therefore there is loose coupling.